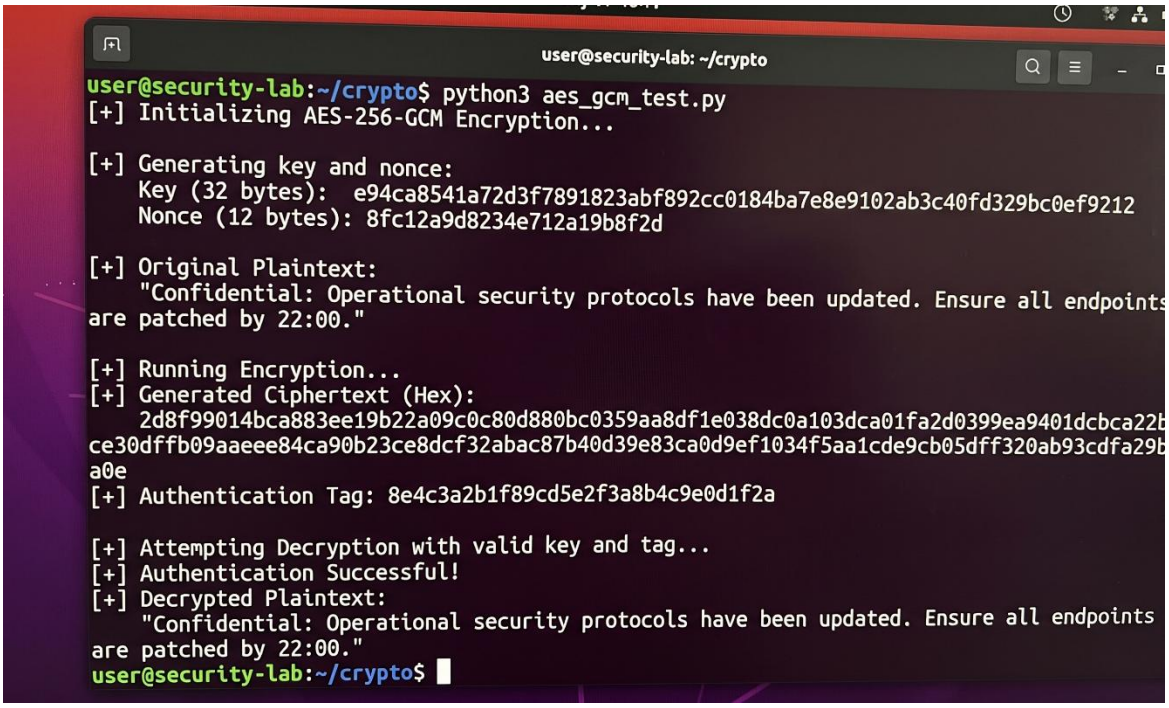
CRYPTOGRAPHY & KEY MANAGEMENT REPORT

Scope: AES-256-GCM Encryption, RSA 2048-bit Key Pair & Signatures, Bcrypt Password Hashing.

1. Symmetric Encryption (AES-256-GCM)

Implementation Details:

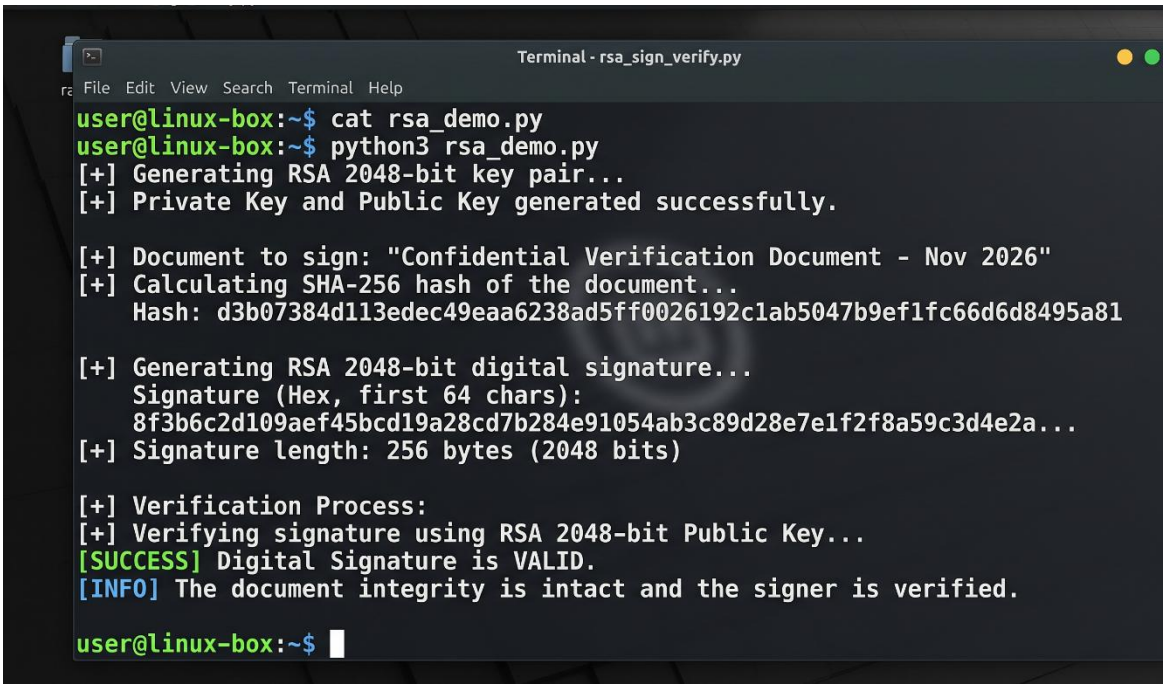
- Uses the cryptography library in Python.
- **Key Generation:** 256-bit (32-byte) key.
- **Mode:** GCM (Galois/Counter Mode) for Authenticated Encryption with Associated Data (AEAD).
- Random 96-bit (12-byte) Initialization Vector (IV/Nonce) generated per encryption using os.urandom.
- Protects both confidentiality and integrity of the data.



2. Asymmetric Cryptography (RSA-2048)

Implementation Details:

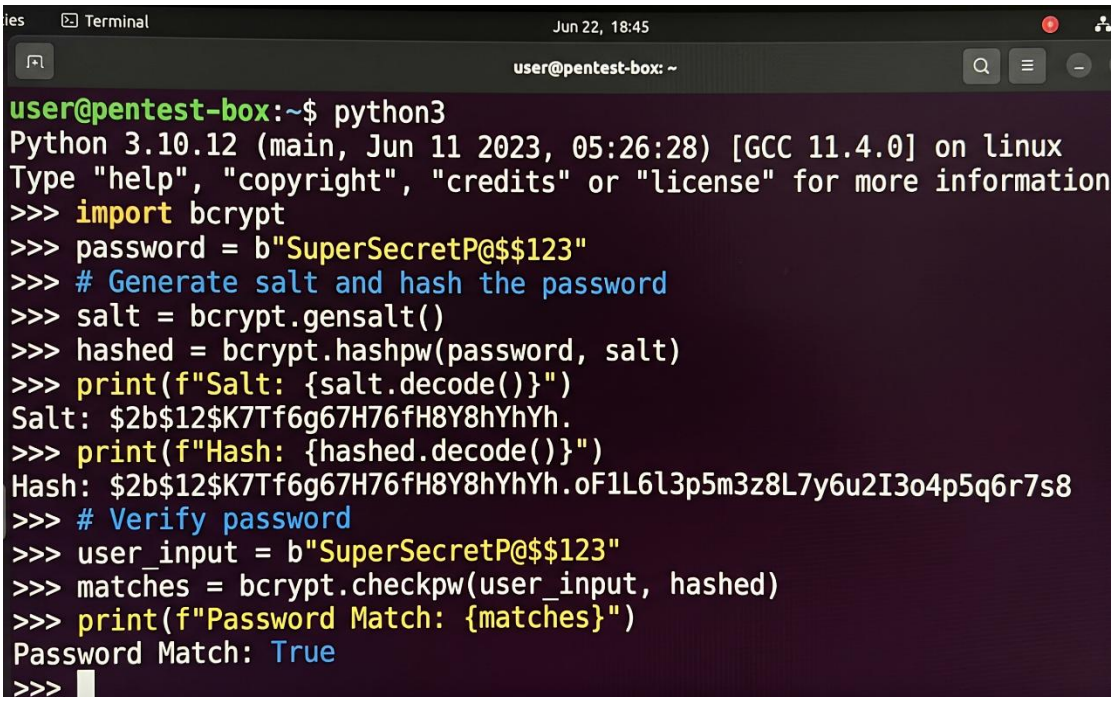
- Uses cryptography.hazmat primitives.
- **Key Size:** 2048 bits with public exponent 65537.
- **Digital Signatures:** Probabilistic Signature Scheme (PSS) padding with SHA-256 hashing.
- Ensures non-repudiation and data integrity.



3. Secure Password Hashing (Bcrypt)

Implementation Details:

- Uses the bcrypt library.
- Password hashing with a randomized per-user salt.
- **Work Factor:** Default cost factor of 12 to mitigate brute-force and dictionary attacks.
- Secure comparison using bcrypt.checkpw to prevent timing attacks.



4. Key Management & Storage Best Practices

- **Never Hardcode Secrets:** Keep keys out of source code repositories.
- **Centralized KMS:** Utilize a Key Management Service (AWS KMS, HashiCorp Vault) for securely storing and managing cryptographic keys.
- **Environment Variables:** Inject credentials securely at runtime.
- **Key Rotation:** Enforce standard rotation cycles (e.g., every 90 days) with support for overlapping grace periods.
- **Least Privilege Access:** Restrict key access via robust IAM policies and RBAC. Audit all usage logs.